

Bancos de dados de XML nativo (Native XML DBMS)



Apresentação

Os bancos de dados de XML nativo têm uma grande importância no contexto empresarial. Diferentes soluções existem no mercado e saber tirar proveito delas será um diferencial competitivo para o profissional em relação aos concorrentes. Muitos aspectos estão ligados à utilização dos bancos de dados de XML nativo, não só técnicos, mas também legais e jurídicos, como a armazenagem de notas fiscais em XML para auditorias da Receita Federal.

Nesta Unidade de Aprendizagem, você vai entender como funcionam os bancos de dados de XML nativo e como as empresas os exploram em suas soluções. Seus conhecimentos serão validados em situações reais e você aprenderá a manipular esses sistemas.

Bons estudos.

Ao final desta Unidade de Aprendizagem, você deve apresentar os seguintes aprendizados:

- Descrever as características dos bancos de dados de XML nativo.
- Explicar o uso de SGBD para armazenar e processar dados com XML nativo.
- Demonstrar o acesso e a manipulação a dados em bancos de XML nativo.



Os bancos de dados de XML nativo são definidos por um modelo lógico para dados de documentos XML. Eles armazenam e recuperam documentos de acordo com esses modelos e têm os documentos XML como sua unidade fundamental de armazenamento. Esses bancos de dados organizam os documentos em coleções, tornando possível a manipulação e consulta de documentos, mantendo-os intactos.

Conheça, no Infográfico a seguir, as funcionalidades, os processos e a arquitetura dos bancos de dados de XML nativo.

BANCOS DE DADOS DE XML NATIVO

São bancos de dados especializados em armazenar documentos XML e manter todos os componentes dos arquivos XML intactos. Esses bancos de dados organizam os documentos em coleções, tornando possível sua manipulação e consulta.

O Native XML Database (NXD) armazena documentos como um conjunto, denominado "COLEÇÃO".



O documento pertencente à coleção recebe um ID ("DOCUMENT IDENTIFIER").



O documento deve ser reutilizável (*round-tripping*), para que a busca retorne o mesmo documento inserido.

FUNCIONALIDADES DE BANCOS DE DADOS DE XML NATIVO



Normalização e integridade referencial

A normalização tem o objetivo de evitar redundância desnecessária no banco de dados e eliminar o risco de dados inconsistentes. A integridade referencial é a verificação de que os documentos XML se referem a documentos válidos ou fragmentos de documentos.



Reutilizável (*round-tripping*)

A reutilização, ou *round-tripping*, significa a capacidade de um banco de dados XML armazenar um documento e depois recuperá-lo sem alterá-lo.



Application Programming Interfaces (APIs)

Os NXDs fornecem pelo menos uma API em linguagens de programação, como Java, C++, C#, etc. Eles fornecem uma interface para conexão ao banco de dados, realizando consultas, recuperando resultados e executando outras tarefas.



Coleções de documentos

Os NXDs organizam documentos em coleções, o que permite consultar e manipular os documentos como um conjunto.



Consulta

No início, a maioria dos NXDs suportava uma linguagem de consulta proprietária. Atualmente, os NXDs têm construído suporte ao XQuery, tornando-se a linguagem de consulta XML de fato.



Atualizações e exclusões

Até o momento, não há um padrão para atualizar dados XML, uma vez que a maioria dos NXDs fornece apenas operações simples de substituição/ exclusão de documentos.

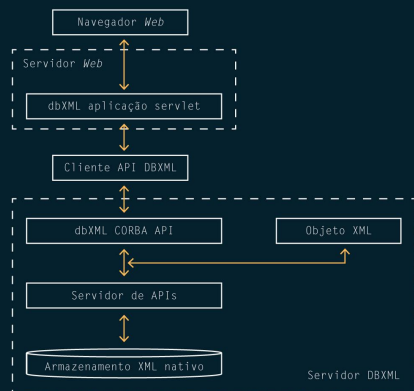


Transações, bloqueio e simultaneidade

Todos os NXDs suportam transações, mas pode diferir o nível em que o bloqueio é implementado. Em sua maioria, o bloqueio ocorre no nível do documento, o que pode levar a problemas na simultaneidade dos usuários.

EXEMPLO: COMUNICAÇÃO APLICAÇÃO WEB X API BANCO DE DADOS DE XML NATIVO

O exemplo a seguir demonstra a leitura de dados em um servidor de banco de dados de XML nativo utilizando API e sendo consumida por uma aplicação Web. Por meio desse exemplo, também é contextualizada a arquitetura CORBA (*Common Object Request Broker Architecture*), uma arquitetura padrão criada pelo *Object Management Group* para estabelecer e simplificar a troca de dados entre sistemas distribuídos heterogêneos.



(Adaptado de Madjarov et al., 2020.)



Aponte a câmera para o código e acesse o link do conteúdo ou clique no código para acessar.



Conteúdo do livro

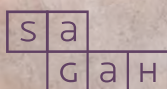
Os bancos de dados de XML nativo são caracterizados como modelos lógicos para dados de documentos XML especializados em armazenar documentos XML e manter os componentes dos arquivos intactos. Esses bancos de dados organizam os documentos em coleções, tornando possível sua manipulação e consulta. Dentre os principais fatores que os diferenciam dos demais bancos existentes, alguns aspectos se tornam relevantes, como: compreender os modelos de armazenamento; entender como funcionam os mecanismos de consulta; saber como projetar a indexação desses bancos.

No capítulo Bancos de dados de XML nativo (*Native XML DBMS*), da obra *Banco de dados não relacional*, base teórica desta Unidade de Aprendizagem, você vai reconhecer a aplicação desses bancos e como eles são operados.

Boa leitura.

BANCO DE DADOS NÃO RELACIONAL

Gabriel Augusto Rosa



SOLUÇÕES
EDUCACIONAIS
INTEGRADAS



Bancos de dados de XML nativo (Native XML DBMS)

Objetivos de aprendizagem

Ao final deste texto, você deve apresentar os seguintes aprendizados:

- Descrever as características de bancos de dados de XML nativo.
- Explicar o uso de SGBD para armazenar e processar dados com XML nativo.
- Demonstrar o acesso e a manipulação a dados em bancos de XML nativo.

Introdução

Os bancos de dados de XML nativos são adotados amplamente por diversas empresas, de modo que é importante que os profissionais da área compreendam suas especificidades e aplicações, para que consigam utilizar e administrar esses bancos. Por meio do uso desses bancos de dados, as empresas podem desenvolver soluções e sistemas que se adequem às suas necessidades.

Neste capítulo, você conhecerá os principais conceitos de bancos de dados de XML nativos. Além disso, verá como se dá a aplicação desses bancos e como eles são operados. Por fim, verá como podem ser aproveitadas as principais vantagens e desvantagens dos bancos de dados XML nativos em relação a outras soluções de banco de dados.

1 Banco de dados de XML nativo: características, funcionalidades e armazenamento

Antes de falar sobre os bancos de dados de XML nativos (NXD, Native XML Database), faz-se necessário entender o que é linguagem XML. De acordo com Oliver (2010), XML (eXtensible Markup Language; ou Linguagem de Marcação Extensível, em português) é uma linguagem de marcação para a criação de documentos com dados organizados hierarquicamente. Ela é classificada como extensível, pois permite definir os elementos de marcação (BRYAN, 1998). Uma linguagem de marcação é um agregado de códigos, que podem ser aplicados a dados para serem lidos por computadores. Para melhor compreensão, é possível fazer uma analogia entre a linguagem XML e a linguagem HTML (HyperText Markup Language; ou Linguagem de Marcação de Hipertexto, em português), pois o XML é utilizado para organizar e formatar uma sequência de dados e separar e integrar o conteúdo a outras linguagens, assim como o HTML, que segue um conceito próximo para a criação de um *website*.

A seguir, são apresentados dois exemplos ilustrativos de arquivos XML. O código a seguir, adaptado de Mello e Barbosa (2018) apresenta um trecho de código exemplificando um esquema XML básico:

```
<raiz>
  <reg>
    <subreg></subreg>
  </reg>
  <reg>
    <subreg>
      <c1>v1</c1>
      <c2>v2</c2>
      <c3>v3</c3>
    </subreg>
  </reg>
</raiz>
```


Já o código a seguir (adaptado de BAX, 2001) apresenta uma descrição de uma ordem de compra:

```
<cliente nome="Fulano de Tal" id="1398">
<OC id="OC150">
  <ITEMOC id="1234">
    <Descricao>Geladeira Islândia Modelo 1234</Descricao>
    <Preco>550,00</Preco>
  </ITEMOC>
  <ITEMOC id="1235">
    <Descricao>Fogão Saara 4 Bocas</Descricao>
    <Preco>450,00</Preco>
  </ITEMOC>
</OC>
</cliente>
```

Um NXD é considerado especializado para armazenar dados XML; ou seja, são bancos de dados capazes de armazenar todos os componentes do modelo XML (elementos, atributos, etc.). Khosrow-Pour (2005) define esses SGBDs (sistemas de gerenciamento de banco de dados) como um modelo lógico para um documento XML, o qual armazena e recupera documentos em formato de coleção de acordo com esse modelo. Contudo, é preciso ressaltar que nem todos os NXDs são bancos de dados realmente independentes, além de não armazenarem o XML na forma nativa verdadeira.

Para Smiljanić *et al.* (2002), esse modelo deve incluir os elementos, os atributos e a ordem do documento, pois, assim como um banco relacional tem uma linha e uma tabela, o banco de dados de XML nativo tem um documento XML como sua unidade fundamental (BRAGANHOLLO, 2011). Todavia, não é necessário ter um modelo de armazenamento físico subjacente específico; ou seja, o NXD pode não ser realmente um banco de dados independente, sendo construído a partir de um banco de dados relacional ou de um formato de armazenamento proprietário, como arquivos compactados e indexados.

A partir das definições apresentadas, pode-se estabelecer as principais características desses SGBDs: armazenagem de documentos XML, gestão de coleções e suporte à consulta em arquivos. Além disso, por serem independentes, eles podem apresentar suas próprias linguagens. Ao relacionar as funcionalidades nativas de um NXD, é preciso esclarecer que nem todos esses SGBDs são iguais. Para Bourret (2010), discutir sobre suas semelhanças e funcionalidades não é uma tarefa simples, uma vez que é possível encontrar

diferentes implementações de produtos de NXD. No entanto, neste capítulo, serão apresentados os principais conceitos que foram aplicados para a criação de produtos de NXD, sendo, portanto, aplicáveis à maioria dos produtos atuais.

Em se tratando de armazenamento XML, pode-se dizer que os NXDs armazenam documentos XML como uma unidade e criam um modelo alinhado com o XML. Esse modelo arbitra níveis de aninhamento, complexidade e suporte para o conteúdo misto, com dados semiestruturados. Todo o modelo é mapeado automaticamente pelo NXD em um mecanismo de armazenamento subjacente. Esse mapeamento garantirá que o modelo específico de dados XML seja mantido.

As principais funcionalidades dos NXDs são apresentadas a seguir (FIGUEIREDO; HIGASI; MURAOKA JUNIOR, 2003).

- **Coleções de documentos:** os NXDs organizam os documentos em coleções, de modo que é possível consultá-los e manipulá-los como um conjunto. As coleções de NXDs não exigem que um esquema seja associado à coleção, porém isso fornece maior grau de flexibilidade no desenvolvimento de aplicativos e aumenta o risco de baixa integridade dos dados.
- **Consulta:** a princípio, a maioria dos NXDs suportava uma linguagem de consulta proprietária. Com o advento do XPath (i.e., uma sintaxe para definir partes de um documento XML, que usa expressões de caminho para navegar em documentos XML), a maioria dos NXDs construiu suporte para ele. No entanto, o XPath não foi projetado para ser uma linguagem de consulta. Nesse sentido, o XQuery (i.e., uma linguagem para localizar e extrair elementos e atributos de documentos XML, construída a partir de expressões XPath; XQuery para XML é como SQL para bancos de dados) acabou se tornando uma recomendação do W3C, e muitos NXDs têm construído suporte a ele, tornando-se uma linguagem de consulta XML de fato.
- **Atualizações e exclusões:** até o presente momento, não há um padrão para atualizar os dados XML, uma vez que a maioria dos NXDs fornece apenas operações simples de substituição/exclusão de documentos. Atualizações sobre o nível do nó são possíveis apenas recuperando um determinado documento, modificando-o usando DOM (Document Object Model) e SAX (Simple API for XML) e retornando-o ao banco de dados.

- **Transações, bloqueios e simultaneidade:** todos os NXDs suportam transações, porém podem diferir quanto ao nível em que o bloqueio é implementado. Na maioria das vezes, o bloqueio ocorre no nível do documento, o que pode levar a problemas na simultaneidade de vários usuários. O problema com o bloqueio no nível do nó é que, para bloquear um nó (n), o pai de n (pn) também precisa ser bloqueado, caso contrário, outra transação poderá excluir pn e, assim, deletar n . Isso ocorre até a raiz do documento, impossibilitando a atualização de outras partes dele que não estão no caminho da raiz para n .
- **API (*application programming interface*; ou programa de aplicação de interface):** os NXDs fornecem pelo menos uma API em linguagens de programação, como Java /C ++ /C #. Eles fornecem uma interface para conectar ao banco de dados, executam consultas, recuperam resultados, entre outras tarefas. A maioria dos NXDs também oferece a capacidade de executar consultas e retornar resultados por HTTP (HyperText Transfer Protocol; ou Protocolo de Transferência de Hipertexto, em português).
- **Reutilizável (*round tripping*):** a reutilização, ou *round-tripping*, indica a capacidade de um banco de dados XML de armazenar um documento e, depois, recuperá-lo sem alteração. Existem vários níveis de *round-trip*: alguns bancos de dados são capazes de oferecer apenas *round-trip* básico, que é para elementos, atributos, texto e hierarquia, ao passo que outros estendem o seu suporte de reutilização para instruções de processamento, comentários e até mesmo estrutura física. Como regra geral, os NXDs baseados em modelo oferecem suporte ao nível de seu modelo, ao passo que os NXDs baseados em texto podem oferecer o suporte total.
- **Normalização e integridade referencial:** a normalização é uma das questões fundamentais da teoria relacional e tem como objetivo evitar a redundância desnecessária em um banco de dados. Como consequência, isso também elimina o risco de dados inconsistentes. Em um NXD, a integridade referencial refere-se à verificação de que os ponteiros em documentos XML indicam documentos válidos ou fragmentos de documentos. O suporte à integridade referencial é oferecido por todos os NXDs, executando uma verificação de validação no esquema sempre que um documento for inserido no banco de dados.



Fique atento

Ao contrário dos arquivos de PDF, o formato XML é um modelo fechado para documentos, de modo que oferece suporte somente para texto puro. Desse modo, são necessários leitores específicos para abrir esse tipo de formato de arquivo. O formato XML é o padrão exigido para as Notas Fiscais Eletrônicas (NF-e), porém os arquivos de notas podem ser abertos pelo leitor disponibilizado pela Receita Federal. É possível encontrar *on-line* leitores de arquivos XML que não requerem instalação, bem como conversores de arquivos XML para outros formatos, como PDF ou outro formato desejado (SUGAI, 2016).

2 Diferentes tipos de bancos de dados de XML nativos

Como diferentes classificações são empregadas a NXDs, é muito importante conhecer as diferenças entre eles. Segundo Powell (2006), existem duas principais classificações: bancos de dados centrados em documentos e centrados em dados. Ambas as formas citadas se referem ao conteúdo, e a distinção entre elas é feita entendendo a natureza dos arquivos XML.

A primeira classificação refere-se a XMLs centrados em documentos, mais próximos à linguagem humana. Nesse tipo de banco de dados, os armazenamentos ocorrem na íntegra e não são acessados sistematicamente ou mesmo pelo conteúdo do elemento XML. Já a segunda classificação refere-se a XMLs centrados em dados, que possuem arquitetura voltada para muitos itens de dados, de forma estruturada e com o objetivo de serem utilizados por páginas da *web* (ELMASRI; NAVATHE, 2018). Um exemplo disso são as páginas de *e-commerce*, em que a flexibilidade e os diferentes tipos de conteúdo fornecem diferentes informações sobre itens comercializados, como livros, por exemplo.

Elmasri e Navathe (2018) explicam que algumas implementações podem ser híbridas, possuindo uma parte estruturada centrada nos dados e outra não estruturada em formato textual. De acordo com Powell (2006, p. 260):

Um tipo especializado de banco de dados XML nativo centrado no documento é chamado de sistema de gerenciamento de conteúdo. Os sistemas de banco de dados de gerenciamento de conteúdo permitem uma certa quantidade de gerenciamento e controle sobre dados XML escritos por humanos, armazenados em tipos de dados XML de bancos de dados XML nativos.

Como os NXDs não requerem uma abordagem generalista, existem diferentes implementações acerca do processamento, da recuperação e da consulta nesses SGBDs. O Quadro 1, a seguir, apresenta diferentes soluções criadas pelos principais fornecedores de NXDs.

Quadro 1. Quadro comparativo de soluções

Fornecedores	Banco de dados	Processo de armazenamento	Recuperação e consulta de XML em SGBDs
Oracle	XML SQL Utility	Armazena o documento XML, mapeando-o para um modelo objeto-relacional, no qual cada elemento XML aninhado é mapeado para o tipo apropriado de objeto. O resultado relacional obtido é uma coleção de tabelas aninhadas.	Correntes de objetos são exploradas ao responder a consultas SQL em bancos de dados relacionais a objetos para produzir o aninhamento de elementos XML na resposta XML final. Expressões XPath podem ser aninhadas nas consultas SQL, permitindo, assim, o uso do XPath como segundo idioma da consulta do SQL.
IBM	DB2	A coluna XML armazena documentos XML inteiros como dados da coluna do DB2. A coleção XML decompõe documentos XML em uma coleção de tabelas relacionais. Definição de tipo de documento (DTD) correspondente ao XML armazenado.	Recria documentos XML usando o procedimento armazenado <code>xxxGenXML()</code> . O parâmetro para esse procedimento é o arquivo DAD. O arquivo DAD pode ser modificado para suportar consultas SQL, gerando, assim, uma exibição arbitrária sobre os dados armazenados.

(Continua)

(Continuação)

Quadro 1. Quadro comparativo de soluções

Fornecedores	Banco de dados	Processo de armazenamento	Recuperação e consulta de XML em SGBDs
IBM	DB2	O documento é armazenado no repositório de DTD e está relacionado com a estrutura de dados XML armazenados por meio do arquivo Data Access Definition (DAD). As ferramentas do XML Extender suportam a criação de arquivo DAD, que é utilizado para destruir e armazenar documentos XML.	
Microsoft	SQL Server	Realiza o armazenamento dos documentos XML em três etapas: 1. um procedimento armazenado <code>sp_xml_preparedocument</code> é chamado no documento XML de origem, criando, assim, uma representação DOM armazenada na memória do documento XML;	Ao adicionar o <code>FOR XML {RAW AUTO EXPLICIT}</code> no final da consulta SQL, o resultado é apresentado como XML. Ao escolher entre <code>RAW</code> , <code>AUTO</code> e <code>EXPLICIT</code> , o nível de controle sobre a estrutura da saída XML é selecionado. Quanto maior o controle, mais complicada será a linguagem para especificar o XML da estrutura de saída.

(Continua)

(Continuação)

Quadro 1. Quadro comparativo de soluções

Fornecedores	Banco de dados	Processo de armazenamento	Recuperação e consulta de XML em SGBDs
Microsoft	SQL Server	2. o mapeamento entre tabela relacional e caminhos XML para elementos atômicos da árvore DOM é estabelecido. Por meio da nova função de conjunto de linhas (semelhante a TABLE ou VIEW OPENXML), os dados DOM são armazenados em tabelas relacionais; 3. o procedimento armazenado sp_xml_removedocument remove o DOM da memória.	O SQL Server tem outra ferramenta, chamada de “Esquemas de mapeamento anotados”, por meio da qual os elementos do esquema XML são anotados com campos relacionais, utilizados para fornecer os dados. O SQL Server pode executar expressões XPath sobre essas anotações-esquemas. Primeiro, ele combina a consulta XPath com o esquema de mapeamento e, depois, gera a resposta, materializando o esquema de mapeamento transformado.

Fonte: Adaptado de Smiljanić *et al.* (2002).

De acordo com Fan *et al.* (2018), em virtude de o NXD não ser uma derivação, mas uma evolução comum a todos os bancos de dados, sejam eles estruturados ou não, também é possível encontrar NXDs distribuídos. Assim como diversas soluções de bancos de dados, as versões distribuídas desses bancos podem realizar a replicação de dados, particionamento vertical e horizontal, reescrita de consulta e otimização de consulta distribuída, envio de consulta/envio de dados, gerenciamento de catálogo distribuído e posicionamento dinâmico de dados.

De acordo com Özsu e Valduriez (2001), a arquitetura e a finalidade que os bancos de dados distribuídos possuem de armazenar dados em múltiplas centrais de processamento melhoram o desempenho no ambiente de trabalho do usuário final, acelerando o processamento das transações nas diversas máquinas, em detrimento de limitar-se a apenas uma. Conforme Moro (2009), as aplicações de NXDs distribuídos têm sido criadas e utilizadas em diversos segmentos, como: *e-commerce*, sobretudo para a integração de informações; *web services* em formato de *feeds* RSS; e em padrões abertos, para a troca de arquivos. Essas aplicações podem ser vistas em diversos segmentos, como, por exemplo, bancos, *marketing* e propaganda, jornalismo, formulários de pesquisas, hotelaria, turismo e passagens de avião.

3 Aplicação prática de banco de dados de XML nativo

Em se tratando de manipular os tipos de NXDs, alguns contextos surgem como principais, são eles: modelos de armazenamento, consulta e indexação. Segundo Papamarkos, Zamboulis e Poulouvassilis (2007), os **modelos de armazenamento** dos NXDs são divididos em dois grupos de abordagem, apresentados a seguir.

- **Armazenamento baseado em texto de XML (*text-based storage*):** essa abordagem armazena os documentos XML como eles são. Por exemplo, dentro de uma única unidade textual.
- **Armazenamento baseado em modelos de XML (*model-based storage*):** esse tipo de abordagem não guarda os arquivos XML como texto, mas sim modela a informação dos documentos XML, transformando-a em uma representação interna de um objeto.

Com relação aos **modelos de consulta** dos NXDs, destaca-se a possibilidade de realizar consultas por conteúdo (busca pelos elementos que contenham determinada palavra) ou estrutura (busca por elementos que são descendentes ou ascendentes ao elemento atual). Ou seja, o mesmo arquivo XML pode ser consultado ou filtrado por conteúdo ou pela estrutura. A seguir, são apresentados exemplos da utilização de um NXD em consulta a uma página da *web* (ELMASRI; NAVATHE, 2018).

A consulta do código a seguir, adaptado de Elmasri e Navathe (2018), traz como resultado os nomes dos empregados com salários acima de R\$ 7.000,00. A variável $\$x$ é atribuída a cada `empregadoNome`, que é filho de um elemento `empregado`, mas apenas para os empregados que satisfazem a restrição de que seu `empregadoSalario` é maior que 7000.

```
FOR  $\$x$  IN
  doc(www.empresa.com/info.xml)
  //empregado [empregadoSalario gt 7000]/empregadoNome
  RETURN <res>  $\$x$ /primeiroNome,  $\$x$ /ultimoNome </res>
```

Já a consulta a seguir, também adaptada de Elmasri e Navathe (2018), apresenta exemplo da aplicação de forma diferente da proposta anterior, para se obter a mesma informação.

```
FOR  $\$x$  IN
  doc(www.empresa.com/info.xml)/empresa/empregado
  WHERE  $\$x$ /empregadoSalario gt 7000
  RETURN <res>  $\$x$ /empregadoNome/primeiroNome,
     $\$x$ /empregadoNome/ultimoNome </res>
```

Para concluir, na consulta a seguir, adaptada de Elmasri e Navathe (2018), a variável $\$x$ está atribuída a cada elemento `projetoEmpregador`, que é filho do projeto número 5, uma vez que a variável $\$y$ está atribuída ao empregado com condição e corresponde aos valores do CPF para recuperar os nomes dos empregados.

```
FOR  $\$x$  IN
  doc(www.empresa.com/info.xml)/empresa
    /projeto[projetoNumero = 5]/projetoEmpregador,
     $\$y$  IN
  doc(www.empresa.com/info.xml)/empresa/empregado
  WHERE  $\$x$ /horas gt 20.0 AND  $\$y$ .cpf =  $\$x$ .cpf
  RETURN <res>  $\$y$ /empregadoNome/primeiroNome,
     $\$y$ /empregadoNome/ultimoNome,  $\$x$ /horas </res>
```

Por meio do uso do modelo de armazenamento baseado em texto, a recuperação de partes de documentos XML é muito mais rápida do que com bancos de dados XML de armazenamento baseado em modelo. Contudo, utilizar este último para realizar algumas alterações, como modificar a ordem de aninhamento dos elementos do documento XML, pode apresentar maior rapidez, se comparado ao modelo baseado em texto. Nesse sentido, não é possível determinar previamente qual é a melhor abordagem a ser utilizada de forma genérica.

Para McHugh e Widom (1999), os índices são utilizados junto às abordagens de armazenamento XML para garantir maior rapidez na recuperação de partes de documentos XML. Nos NXDs, um dos **modelos de indexação** existentes e uma das estruturas clássicas é utilizar um arquivo invertido para pesquisa de palavras-chave, o qual é estruturado como uma lista de pares <word, document>, para que seja localizado em um documento. Arquivos invertidos separados podem ser utilizados para indexar *tags* XML e conteúdo textual.

Para finalizar a compreensão sobre os NXDs, é importante conhecer as vantagens e desvantagens do SGBDs XML nativos. Como principais vantagens dos NXDs, pode-se destacar que eles têm maior capacidade de armazenar, manter e consultar grandes quantidades de documentos XML do que os bancos de dados relacionais, e, ao contrário destes, nenhuma tabela precisa ser configurada e nenhum *design* complicado precisa ser feito antes da configuração do banco de dados. Contudo, existem algumas desvantagens no SGBD, como o fato de os bancos de dados relacionais estarem bem-estabelecidos no mercado e já serem uma tecnologia comprovada, ao passo que os bancos de dados XML são mais recentes e vistos por alguns como uma experiência limitada.



Saiba mais

Os principais problemas encontrados nos bancos de dados XML são apresentados a seguir.

- **Vários níveis de validade:** um documento XML é válido ao testar a conformidade do arquivo para um esquema DTD ou XML específico. Esse tipo de banco deve, pelo menos, distinguir entre três níveis básicos de validade: dados não XML, XML bem formado e XML válido, documentos que confirmam o esquema XML fornecido pelo aplicativo ou o esquema XML armazenado internamente no banco de dados XML.

- **Entidades e URI (*uniform resource identifier*; ou identificador uniforme de recurso):** o banco de dados XML deve estar ciente de que os elementos XML podem se referir às entidades fora do banco de dados no qual eles estão armazenados. Isso pode violar a independência dos dados, expondo algumas decisões de *design* relacionadas com o posicionamento dos dados, sobretudo se o usuário estiver forçado a incluir o URI, com o objetivo de localizar os dados na recuperação de dados Query.
- **Transformações:** o XSLT é comumente utilizado para transformar um documento XML em outro, de formulário desejado. Os bancos de dados XML devem poder gerenciar os programas XSLT e executá-los em documentos XML, além de permitir a criação de *scripts* de consulta e/ou transformação arbitrária, permitindo, assim, a montagem complexa de documentos XML.



Referências

- BAX, M. P. Introdução às linguagens de marcas. *Ciência da Informação*, Brasília, v. 30, n. 1, p. 32–38, jan./abr. 2001. Disponível em: https://www.scielo.br/scielo.php?script=sci_abstract&pid=S0100-19652001000100005&lng=pt&nrm=iso&tlng=pt. Acesso em: 20 jun. 2020.
- BOURRET, R. XML Database Products. *Ronald Bourret*, Felton, 2010. Disponível em: <http://www.rpbouret.com/xml/XMLDatabaseProds.htm>. Acesso em: 20 jun. 2020.
- BRAGANHOLLO, V. *Armazenamento nativo*. Niterói: Instituto de Computação, Universidade Federal Fluminense, 2011. 41 p. (Notas de aula da disciplina Gerência de Dados Semi-Estruturados). Disponível em: <http://www2.ic.uff.br/~vanessa/material/gdse/06-ArmazenamentoNativo.pdf>. Acesso em: 20 jun. 2020.
- BRYAN, M. An Introduction to the Extensible Markup Language (XML). *Bulletin of the American Society for Information Science and Technology*, Silver Spring, v. 25, n. 1, p. 11–14, Oct./Nov. 1998. Disponível em: <https://asistdl.onlinelibrary.wiley.com/doi/full/10.1002/bult.104>. Acesso em: 20 jun. 2020.
- ELMASRI, R.; NAVATHE, S. B. *Sistemas de banco de dados*. 7. ed. São Paulo: Pearson, 2018. 1152 p.
- FAN, H. *et al.* Handling distributed XML queries over large XML data based on MapReduce framework. *Information Sciences*, [S. l.], v. 453, p. 1–20, July 2018.
- FIGUEIREDO, F. J. V.; HIGASI, H.; MURAOKA JUNIOR, L. M. *XML e bancos de dados*. Orientador: Rogério de Oliveira. 2003. 94 f. Monografia (Bacharelado em Sistemas de Informação) – Faculdade de Computação e Informática, Universidade Presbiteriana Mackenzie, São Paulo, 2003. Disponível em: <http://meusite.mackenzie.com.br/rogerio/tgi/2003XMLIndex.PDF>. Acesso em: 20 jun. 2020.

KHOSROW-POUR, M. (ed.). *Encyclopedia of information science and technology*. Hershey: IGI Global, 2005. 3807 p.

MCHUGH, J.; WIDOM, J. Query optimization for XML. In: International Conference on Very Large Data Bases, 25., 1999, Edinburgh. *Proceedings* [...]. Edinburgh: Napier University, 1999. Disponível em: <http://ilpubs.stanford.edu:8090/374/>. Acesso em: 20 jun. 2020.

MELLO, C. L.; BARBOSA, D. M. *XML nativo como alternativa ao modelo relacional clássico*. Florestal: Universidade Federal de Viçosa, 2018. 14 p. Disponível em: <http://www.novoscurso.ufv.br/graduacao/caf/ccp/www/wp-content/uploads/2019/02/POC-II-2018-2-Caio-de-Lima-Mello.pdf>. Acesso em: 20 jun. 2020.

MORO, M. M. XML Nativo. In: SIMPÓSIO BRASILEIRO DE BANCO DE DADOS, 23., 2009, Campinas. *Mini-cursos* [...]. Campinas: Centro de Convenções da Unicamp, 2009. Disponível em: <https://homepages.dcc.ufmg.br/~mirella/pdf/Slides-Minicurso.pdf>. Acesso em: 20 jun. 2020.

OLIVER, P. R. C. *Projetos de ECM/BPM: os segredos da construção*. São Paulo: Biblioteca24horas, 2010. v. 2. 292 p.

ÖZSU, M. T.; VALDURIEZ, P. *Princípios de sistemas de bancos de dados distribuídos*. 2. ed. Rio de Janeiro: Campus, 2001. 711 p.

PAPAMARKOS, G.; ZAMBOULIS, L.; POULOVASSILIS, A. *XML databases*. London: School of Computer Science and Information Systems, Birkbeck College, University of London, 2007. 20 p. Disponível em: <http://students.mimuw.edu.pl/~pd291528/zbd/materialy/Xml%20Databases.pdf>. Acesso em: 20 jun. 2020.

POWELL, G. *Beginning XML databases*. Indianapolis: Wiley, 2007. 470 p.

SMILJANIĆ, M. et al. *Distributed XML database systems*. Twente: Database group, Faculty of Informatics, University of Twente, 2002. 43 p. Disponível em: <https://ris.utwente.nl/ws/portalfiles/portal/28108011/000000b0.pdf>. Acesso em: 20 jun. 2020.

SUGAI, A. O que é e como abrir um arquivo xml? *TechTudo*, Rio de Janeiro, 13 nov. 2016. Disponível em: <https://www.techtudo.com.br/noticias/noticia/2016/11/o-que-e-e-como-abrir-um-arquivo-xml.html>. Acesso em: 20 jun. 2020.

Leituras recomendadas

NATIVE XML databases. *XML4Pharma*, Thal, 2012. Disponível em: <http://www.xml-4pharma.com/XMLDB/>. Acesso em: 20 jun. 2020.

POKORNÝ, J. XML in enterprise systems: its roles and benefits. In: BERNUS, P.; DOUMINGOS, G.; FOX, M. (ed.). *Enterprise architecture, integration and interoperability*. Berlin; Heidelberg; New York: Springer, 2010. p. 128–139. Disponível em: https://link.springer.com/content/pdf/10.1007/978-3-642-15509-3_12.pdf. Acesso em: 20 jun. 2020.



Fique atento

Os *links* para *sites da web* fornecidos neste capítulo foram todos testados, e seu funcionamento foi comprovado no momento da publicação do material. No entanto, a rede é extremamente dinâmica; suas páginas estão constantemente mudando de local e conteúdo. Assim, os editores declaram não ter qualquer responsabilidade sobre qualidade, precisão ou integralidade das informações referidas em tais *links*.

Encerra aqui o trecho do livro disponibilizado para esta Unidade de Aprendizagem. Na Biblioteca Virtual da Instituição, você encontra a obra na íntegra.

Conteúdo:



SOLUÇÕES
EDUCACIONAIS
INTEGRADAS



Dica do professor

As otimizações em bancos de dados têm a intenção de fornecer a garantia contínua do desempenho de soluções, para que tenham as condições necessárias para uma boa performance em sistemas. As organizações têm focado seus esforços na aplicação de otimizações.

Veja, nesta Dica do Professor, algumas dicas sobre o tema e, também, sugestões de ações que podem auxiliar no ganho de desempenho.



Aponte a câmera para o código e acesse o link do conteúdo ou clique no código para acessar.



Saiba mais

Para ampliar o seu conhecimento a respeito desse assunto, veja abaixo as sugestões do professor:

A análise da modelagem e consistência dos dados na era do *Big Data*

Este artigo contextualiza os bancos de dados de XML nativo como fonte de armazenamento de dados semiestruturados, também comparando a modelagem e a consistência de dados XML com outros formatos de dados.



Aponte a câmera para o código e acesse o link do conteúdo ou clique no código para acessar.

XML nativo como alternativa ao modelo relacional clássico

Este artigo propõe a utilização de bancos de dados de XML nativo como alternativa a bancos de dados relacionais, comparando-os e diferenciando-os.



Aponte a câmera para o código e acesse o link do conteúdo ou clique no código para acessar.

Comparativo de desempenho entre sistemas gerenciadores de banco de dados para armazenamento de dados XML em formato nativo

Este artigo traz dados sobre parâmetros para um bom desempenho de um banco de dados de XML nativo.



Aponte a câmera para o código e acesse o link do conteúdo ou clique no código para acessar.

Uma abordagem alternativa para o armazenamento electrónico de dados: o papel da norma XML

Este artigo traz informações sobre as diferentes formas de armazenamento e gestão de dados XML existentes atualmente.



Aponte a câmera para o código e acesse o link do conteúdo ou clique no código para acessar.